Deep Learning on CPUs in Heterogeneous Computing Scenario

Sparsh Mittal

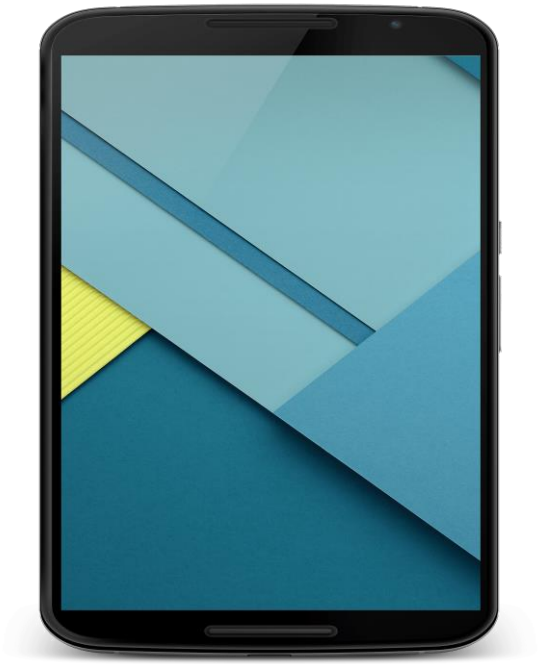
Contents

- Wearable+handheld collaborative computing.
- Mobile+cloud collaborative computing.
- CPU+GPU/DSP collaborative computing

Acronyms

- LSTM-HAR – Long-short term memory-human activity recognition
- DNN – Deep neural network
- MNIST – Modified National Institute of Standards and Technology dataset
- RNN – Recurrent neural network
- ML – Machine learning
- LTE – Long term evolution
- TOPS – Tera operations

Wearable-Handheld Collaborative Computing



This discussion is based on following paper: M. Xu *et al.*, "DeepWear: Adaptive local offloading for on-wearable deep learning," *IEEE Transactions on Mobile Computing*, vol. 19, no. 2, pp. 314-330, Feb. 2020. The images and tables are courtesy of this paper.

Experimental Platform

Hardware

Device	CPU	Memory	GPU
Nexus 6	Quad-core Krait 450	3 GB RAM	Adreno 420
LG Urbane	Quad-core Cortex-A7	512MB RAM	Adreno 305
Galaxy S2	Dual-core Cortex-A9	1GB RAM	Mali-400MP4

Benchmarks

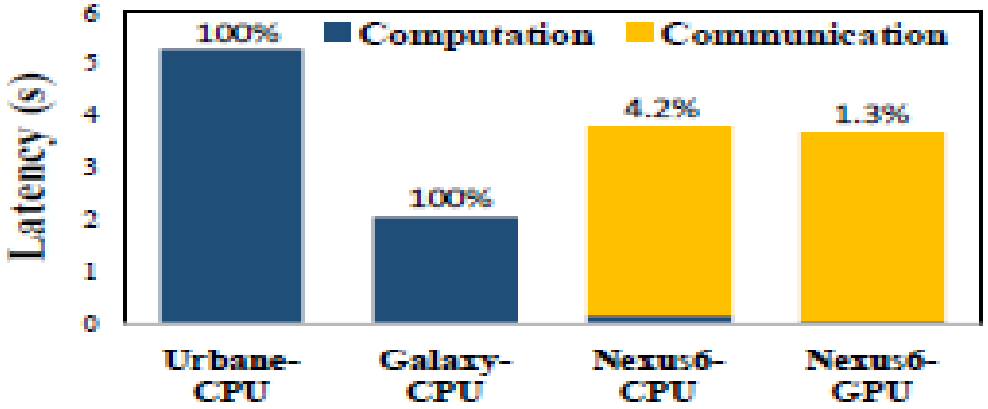
Model	App	Input	FLOPs
MNIST [41]	digit recognition	grayscale image	15M
MobileNet [30]	image classification	rgb image	580M
GoogLeNet [42]	image classification	rgb image	2G
LSTM-HAR [43]	activity recognition	mobile sensor	180M
DeepSense [44]	activity recognition	mobile sensor	550M
TextRNN [45]	document classification	word vectors	11M
DeepEar [6]	emotion recognition	raw sound	9M
WaveNet [46]	speech recognition	mfcc features	3.8G

Scenario

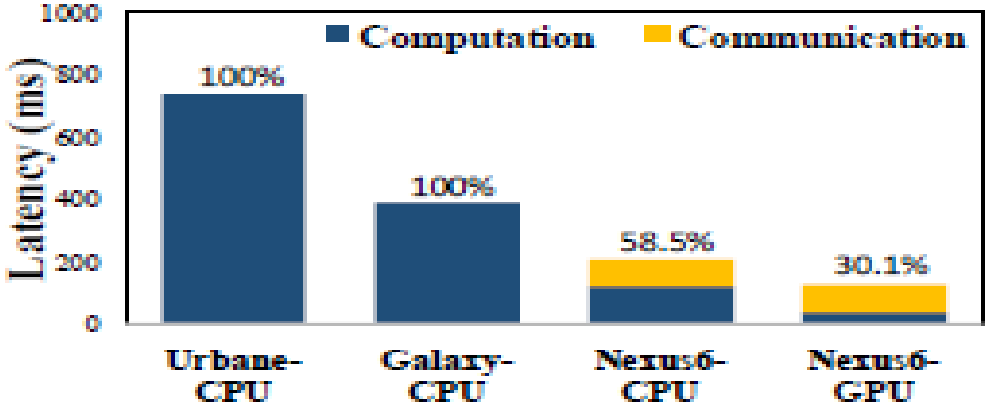
- Wearable: LG Watch Urbane (with 410 mAh battery).
- Handheld: Nexus 6 smartphone (with 3220 mAh battery).
- Data transfer between them is done using Bluetooth.
- On handheld, a DNN model is run on either CPU or GPU.
- Default governor: interactive, runs at high frequency.
- Conventional techniques:
  1. No offloading: run on wearable.
  2. Complete offloading: run on handheld.
- Both these are inefficient, as we show next

Impact of Data Transfer Overheads on Latency

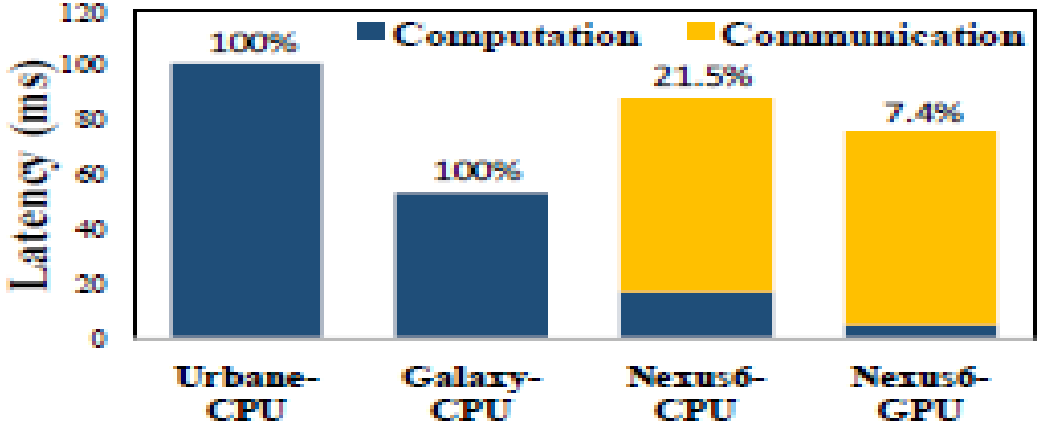
- Although handheld can reduce the compute latency, the overall latency may not decrease due to the high latency of data transfer.



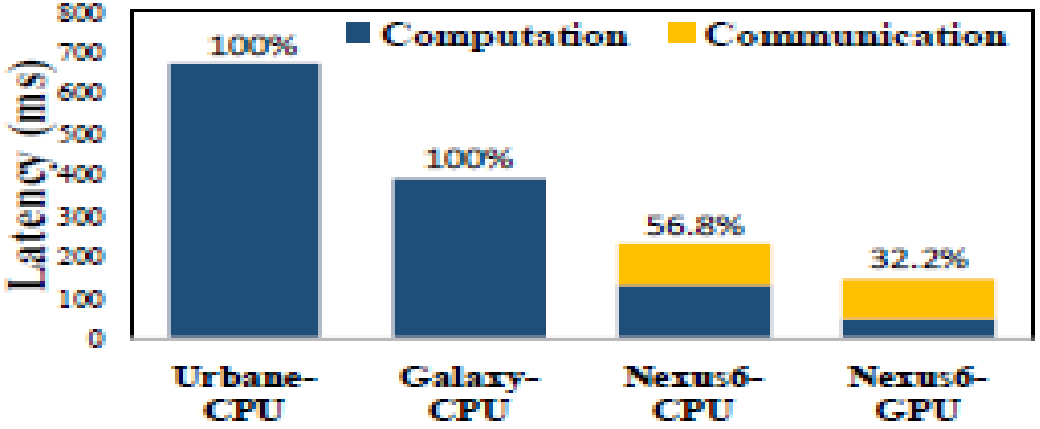
(a) *GoogLeNet* model.



(b) *LSTM-HAR* model.



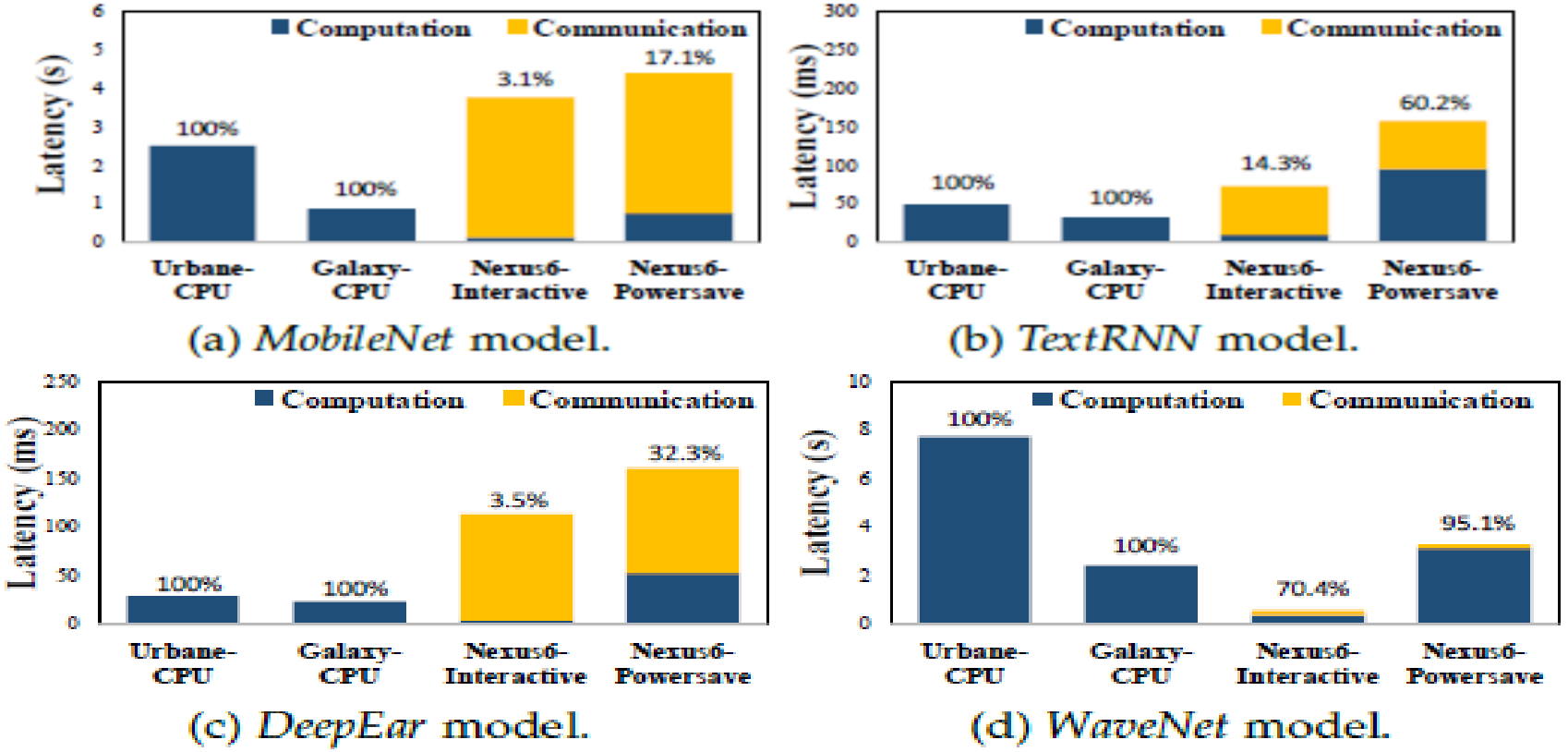
(c) *MNIST* model.



(d) *DeepSense* model.

Impact of Energy Governor on Latency

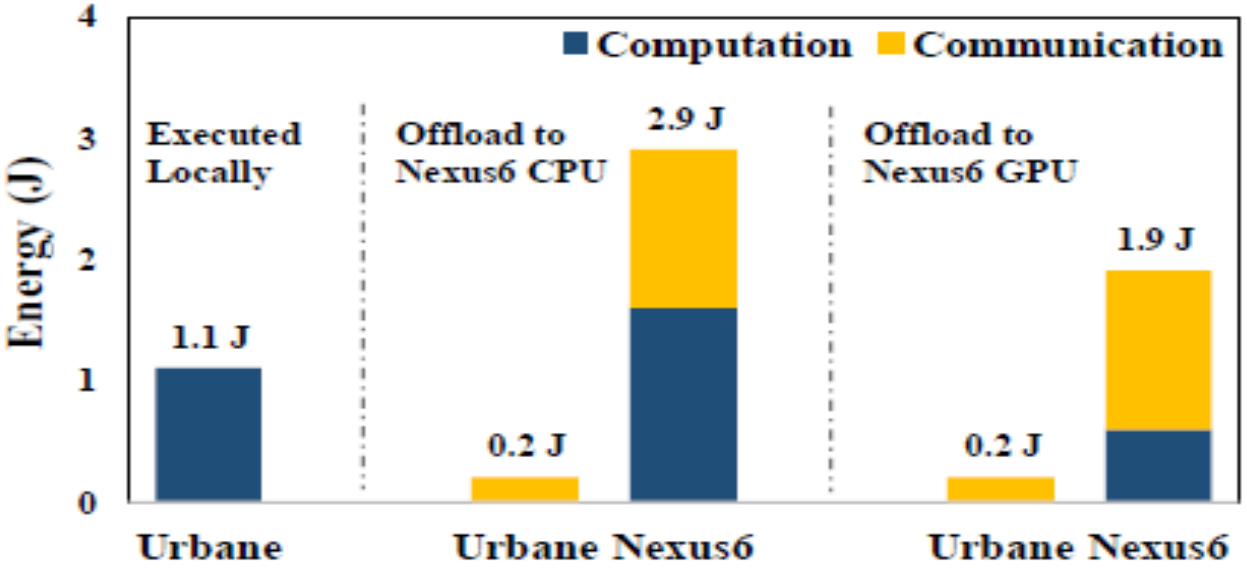
- Powersave governor runs handheld at low frequency when battery is low or screen is off.
- If handheld is managed by the powersave governor, latency may be significantly increased.



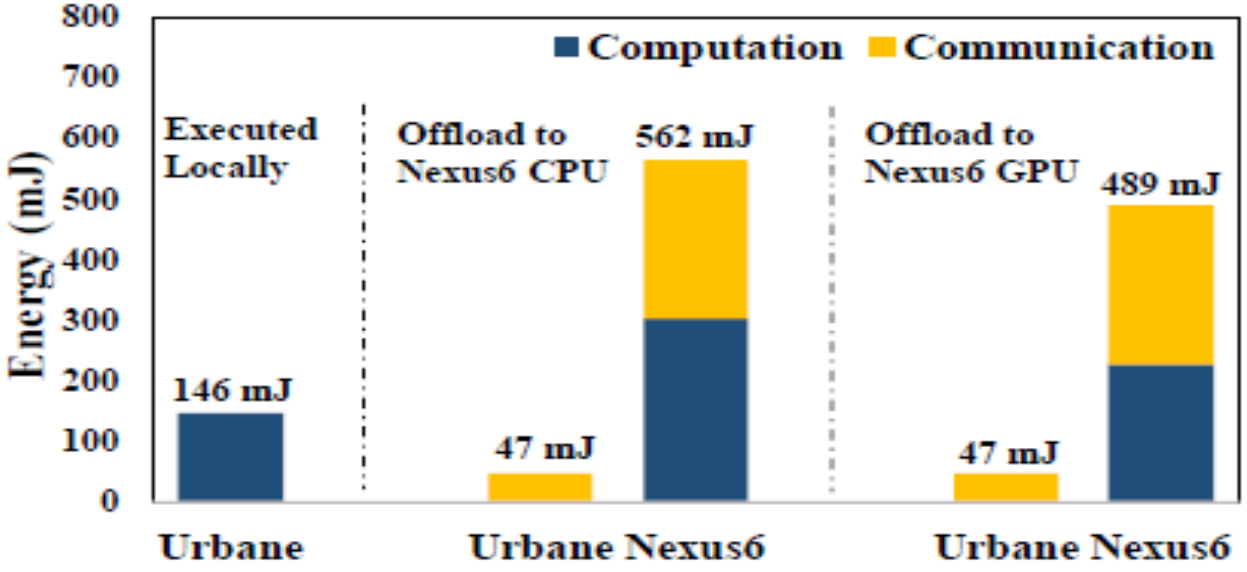
(a) *MobileNet* model. (b) *TextRNN* model. (c) *DeepEar* model. (d) *WaveNet* model.

Impact of Data Transfer on Energy

- More offloading increases the total energy consumption of wearable+handheld due to Bluetooth energy.



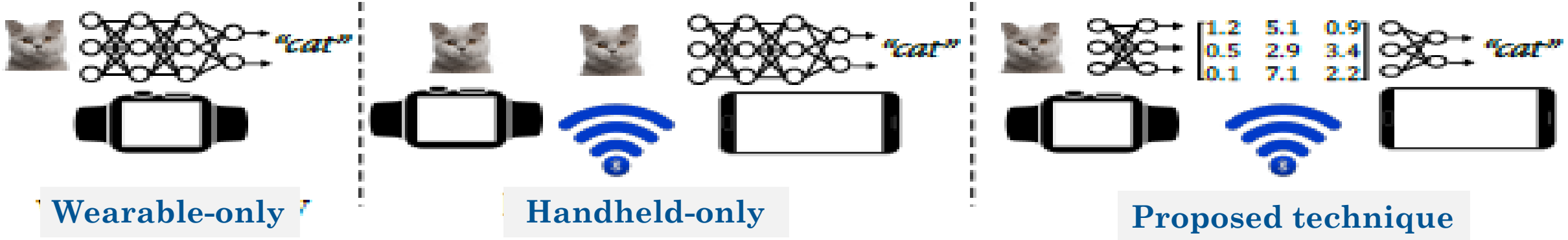
(a) *GoogLeNet* model.



(b) *LSTM-HAR* model.

Proposed Technique: Partial Offloading

- Compute some layers on wearable and remaining layers on paired handheld.



- They use ML models to find latency and energy of different layers on different devices.
- Based on this, a DNN model is partitioned between wearable and handheld.

Results

- Their model also accounts for Bluetooth bandwidth, processor load, battery status, pairing status, and so on.
- Their technique saves the total energy of handheld+wearable.
- Reduces latency because the output of an internal layer is smaller than that of the input layer.
- Reduces data-transfer overheads.
- Furthermore, wearable and handheld can do pipelined execution.

Mobile+Cloud Collaborative Computing

This discussion is based on the following paper: Y. Kang *et al.*, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 615–629. The images are courtesy of this paper

Experimental Platform

Platforms

- Mobile CPU
- Mobile GPU
- Cloud GPU

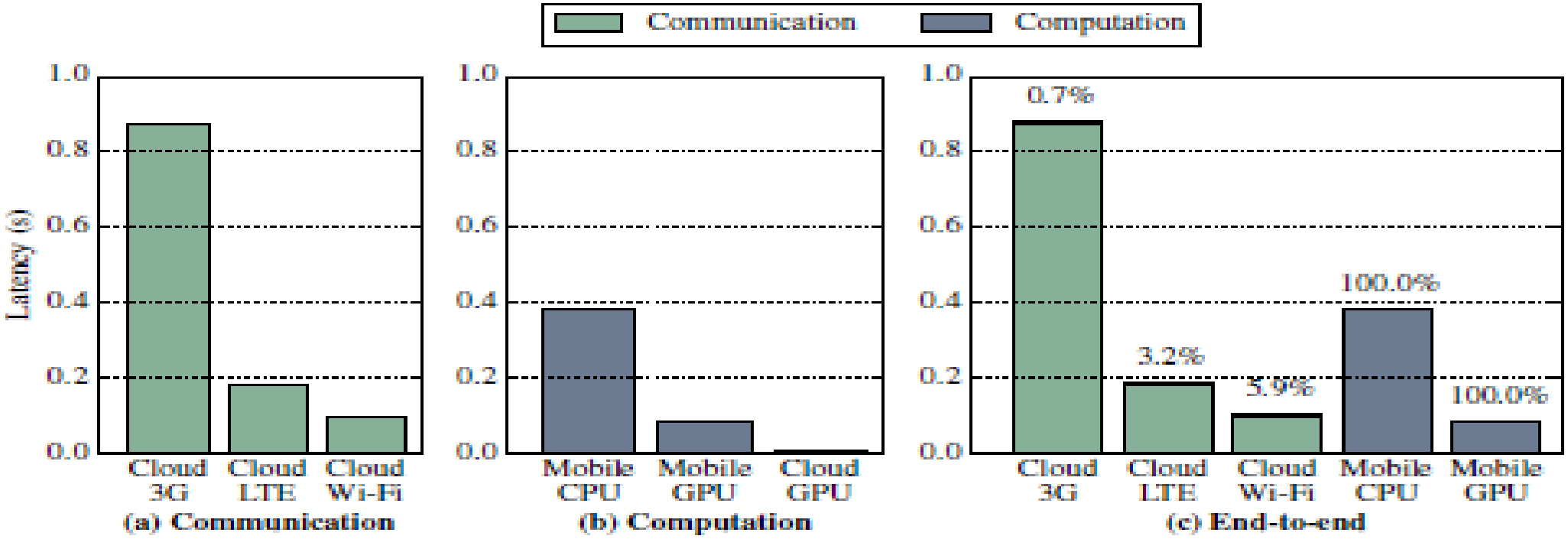
Increasing performance

Increasing speed

Networks

- 3G
- LTE
- WiFi

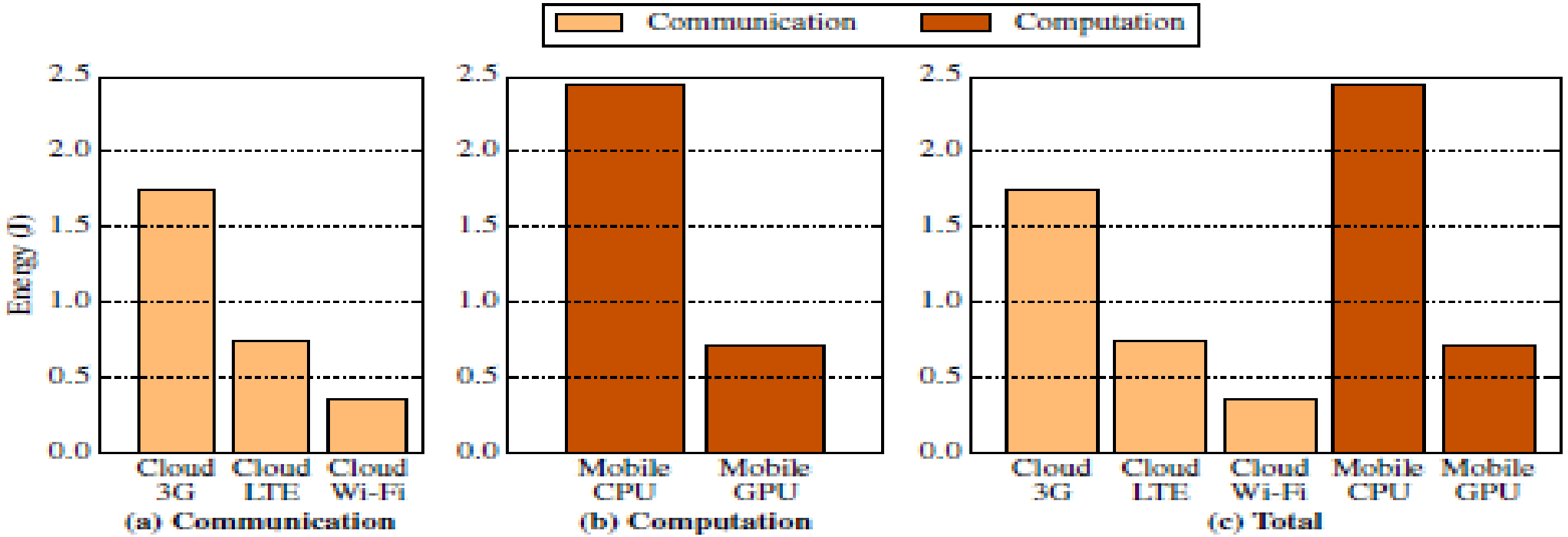
Observations



Latency with mobile CPU is much lower than that of data transfer via 3G.

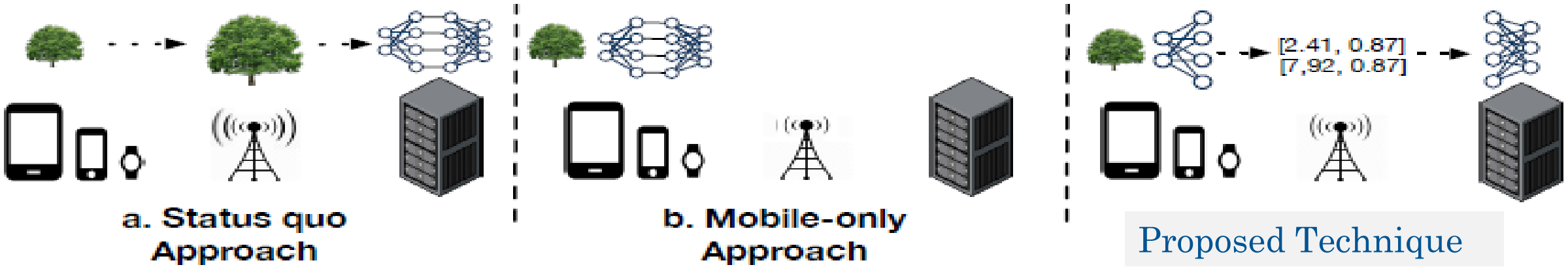
Observations

- Inference using mobile GPU is more energy efficient than data transfer over 3G or LTE.
- Cloud-only execution is superior to using WiFi.
- Otherwise, mobile-only execution provides superior performance and energy efficiency.



Proposed Technique

- Partitions a DNN between mobile and cloud.



- Optimizes overall latency or mobile energy.
- Takes into account load at cloud.
- Improves data-center throughput and allows the data center to handle more user queries.

CPU+GPU heterogeneous computing on mobile devices

Lane et al., "DeepX: A Software Accelerator for Low-Power Deep Learning Inference on Mobile Devices", IPSN, 2016

Motivation and Proposed technique

- **Motivation:** Running entire model on a single processor creates memory bottlenecks.
- **Idea:** They compress selected layers of DNN at runtime using singular value decomposition.
- It causes only small loss in accuracy.

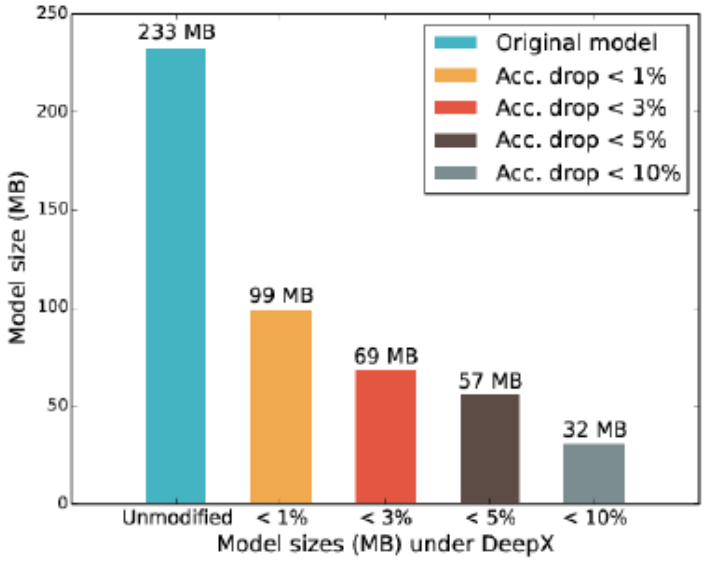


Fig. 9: Memory requirement of AlexNet under DeepX

Decompose the DNN into different parts that can be executed at CPU and GPU/DSP such that performance and resource usage are maximized.

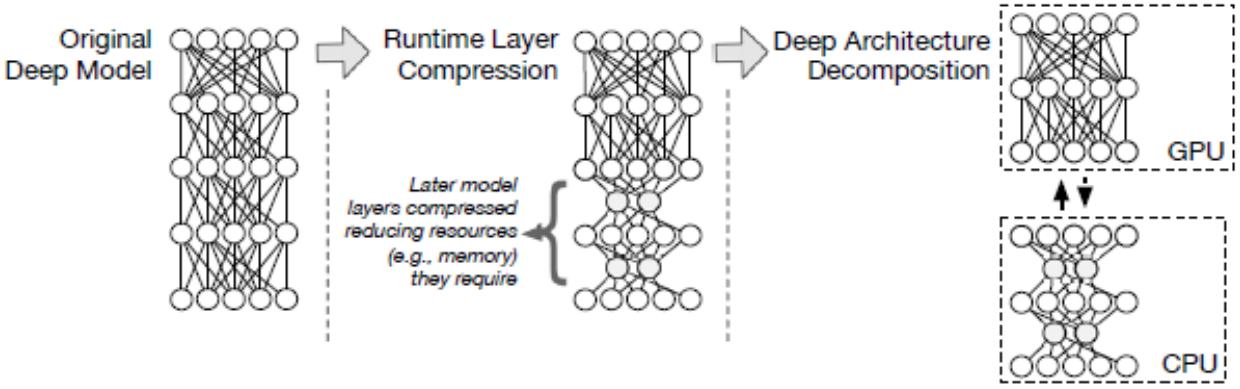


Fig. 2: Representative example of model decomposition and compression in operation under DeepX

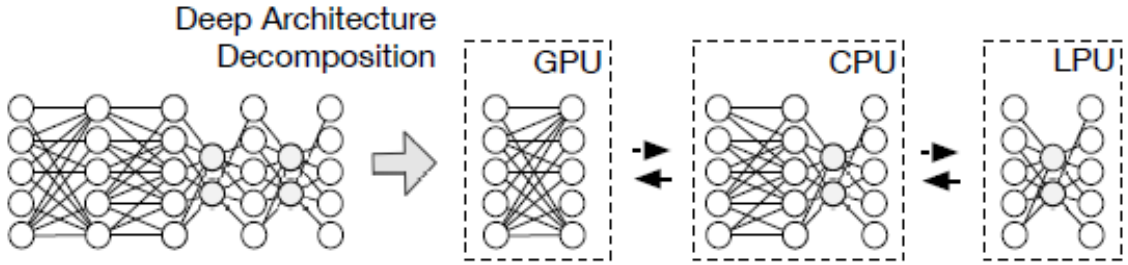


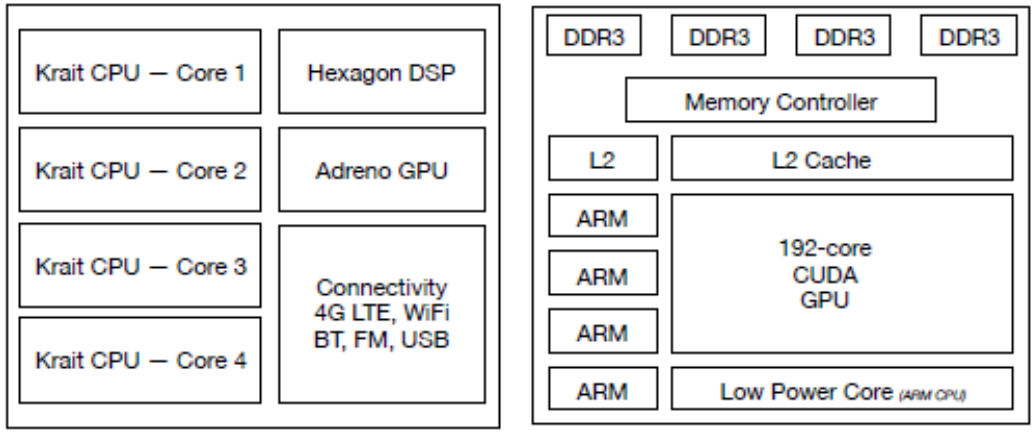
Fig. 5: Illustration of deep architecture decomposition. Different parts of the model architecture are assigned to available computational resources for runtime efficiency.

Insights

- Compressed layers are executed on CPU.
  - Compression reduces their memory and computation demands.
- Remaining layers are run on GPU.
- Decomposition engine accounts for layer-dependencies, the resources of CPU/GPU, current resource usage status and overhead of recomposition.
- Decomposition is modeled as a “mixed-integer linear programming” and solved using a CPLEX solver

Experiments

- DNNs: AlexNet, SpeakerID, SVHN and AudioScene
- On two SoCs
  - **NVIDIA TK1** has a GPU, a four-core CPU, and a 5th core, which is a low-power core.
  - **Qualcomm Snapdragon** 800 has CPU and DSP



(a) Snapdragon 800 (b) Tegra K1

Their technique achieves lower latency and energy consumption than running the DNN model exclusively on CPU, GPU, DSP, or in the cloud.

Relative energy consumption normalized to DeepX technique

	CPU *(only) (mJ)*	DSP *(only) (mJ)*	Cloud *(only) (mJ)*
AlexNet	933.5 (2.1×)	–	4978.4 (11.2×)
SVHN	230.9 (2.6×)	142.1 (1.6×)	1101.1 (12.4×)
SpeakerID	113.4 (8.1×)	103.6 (7.4×)	124.2 (8.9×)
AudioScene	110.3 (8.0×)	99.3(7.2×)	122.7 (8.9×)

(a) Qualcomm Snapdragon 800 SoC

	CPU *(only) (mJ)*	LPU *(only) (mJ)*	GPU *(only) (mJ)*	Cloud *(only) (mJ)*
AlexNet	1681.3 (13.2×)	–	234.1 (1.8×)	2820 (22.1×)
SVHN	479.6 (4.3×)	–	167.3 (1.5×)	1382.9 (12.4×)
SpeakerID	7.1 (7.8×)	109.1 (120.4×)	1.3 (1.4×)	26.9 (29.7×)
AudioScene	6.7 (7.6×)	106.1 (120.3×)	1.2 (1.4×)	26.1 (29.4×)

(b) Nvidia Tegra K1 SoC

Why their technique is effective

- Their technique avoids memory bottleneck by dividing the layers and utilizing the idle processor such as DSP.
- When the deadline of inference is large enough,
  - Put power-hungry GPU to sleep
  - Perform inference on one or more low-power core(s) to save energy.